ROSMASTER CRASH COURSE

# What is ROS 2?

Week 1 of 4. From zero to a robot that drives itself.

**Phan Phearamony**
KAMAL Laboratory, Gunma University



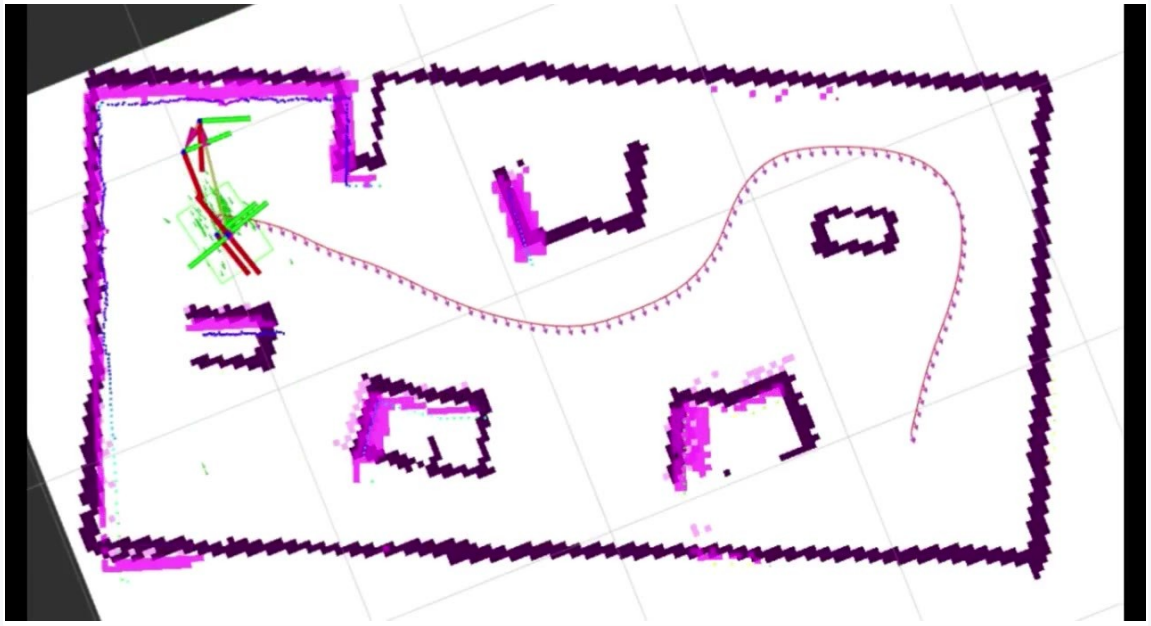
Yahboom ROSMASTER A1

THE GOAL

# By Week 4, you will understand how a robot moves like this



Real ROSMASTER A1



What the robot sees (RViz2)

**Known map** — Surprise obstacles — Unknown room

## Scenario 1: Known map

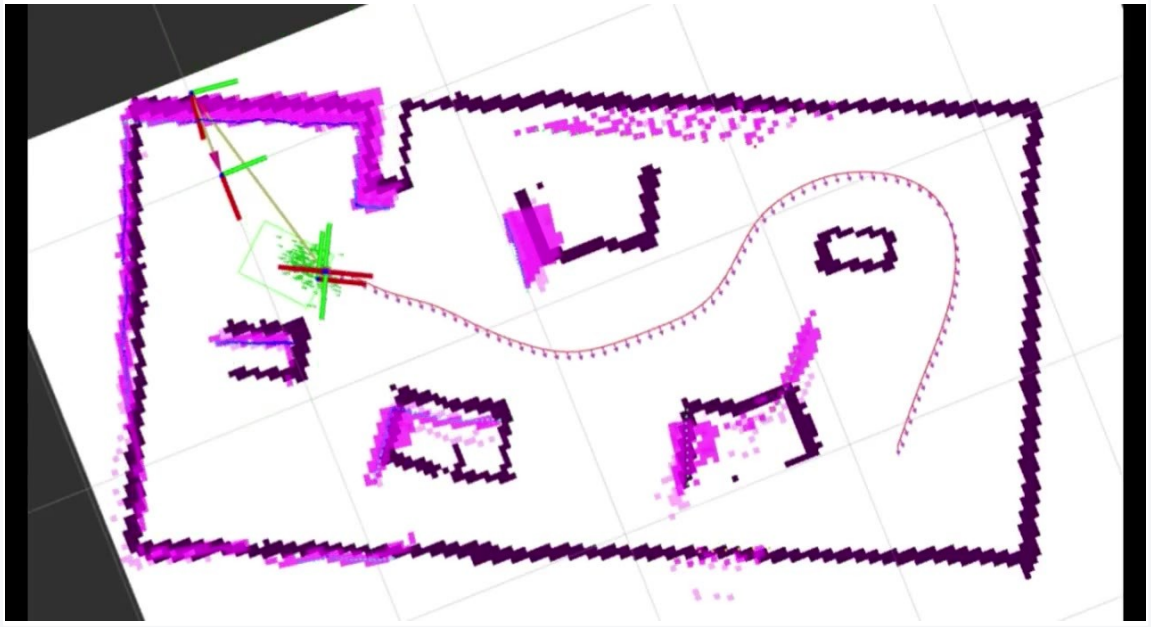
The robot already has a map of the room. It plans a path and drives to the goal.

THE GOAL

# By Week 4, you will understand how a robot moves like this



Real ROSMASTER A1



What the robot sees (RViz2)

Known map — **Surprise obstacles** — Unknown room

## Scenario 2: Surprise obstacles

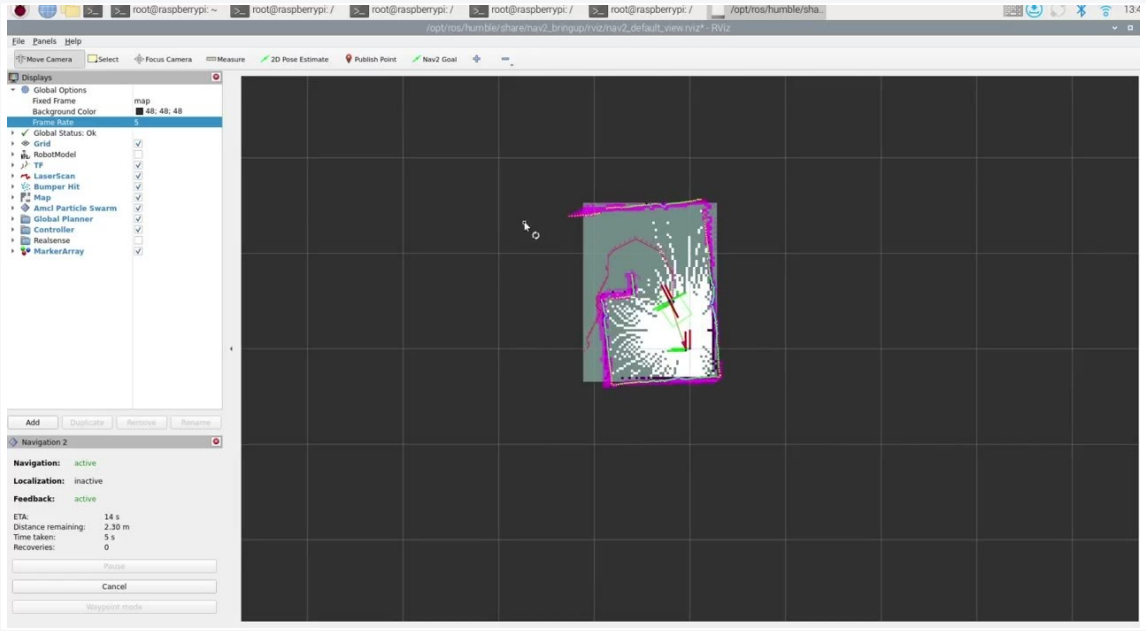
Same map, but we add boxes the map does not know about. The robot sees them with LiDAR and goes around.

THE GOAL

# By Week 4, you will understand how a robot moves like this



Real ROSMASTER A1




What the robot sees (RViz2)

Known map — Surprise obstacles — **Unknown room**

## Scenario 3: Unknown room

No map at all. The robot builds the map while it drives, and still finds the goal.

COURSE ROADMAP

# Four weeks, one robot

WEEK 1
**ROS 2 Introduction**

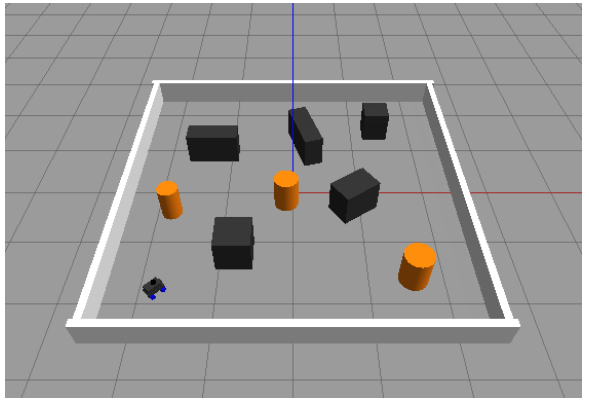
WEEK 2
Reading the Code

WEEK 3
Control and Sensors

WEEK 4
ROSMASTER Functions

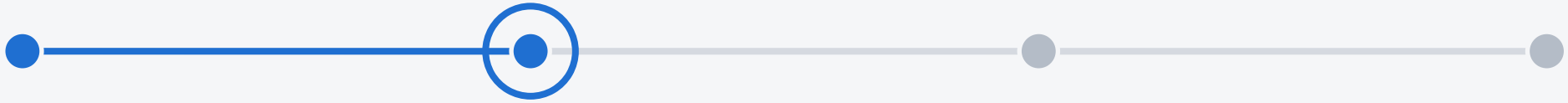
## 01

- What ROS 2 is and why robots use it
- Nodes, topics and messages
- ROS vs ROS 2, and the versions
- Gazebo, the robot simulator
- Hands-on: a simulated A1 avoids obstacles with LiDAR



COURSE ROADMAP

# Four weeks, one robot



WEEK 1
ROS 2 Introduction

WEEK 2
**Reading the Code**

WEEK 3
Control and Sensors

WEEK 4
ROSMASTER Functions

## 02

- Linux: files, folders and the terminal
- Bash commands you use every day: cd, ls, nano, source, sudo apt
- Python basics: variables, if, functions, classes
- Read the Week 1 avoidance node line by line

```
def scan_callback(...):
    front = ...
    if front < safe:
        turn()
```

COURSE ROADMAP

# Four weeks, one robot

WEEK 1
ROS 2 Introduction

WEEK 2
Reading the Code

WEEK 3
**Control and Sensors**

WEEK 4
ROSMASTER Functions

**03**

- How the A1 moves: /cmd_vel, Ackermann steering, odometry
- Its sensors: LiDAR, camera, IMU, wheel encoders
- Example: LiDAR obstacle avoidance on the real A1, like Week 1
- Homework: use the A1's built-in AI (YOLO) to recognize objects



COURSE ROADMAP

# Four weeks, one robot

WEEK 1
ROS 2 Introduction

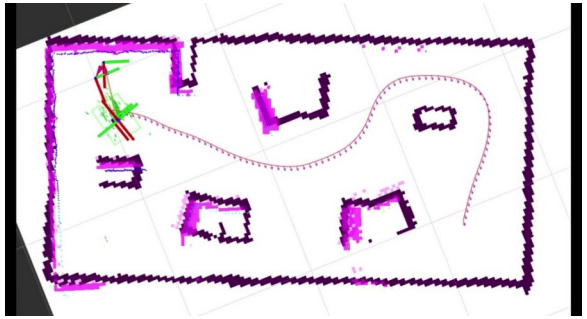
WEEK 2
Reading the Code

WEEK 3
Control and Sensors

WEEK 4
**ROSMASTER Functions**

## 04

- Tour the A1's built-in functions from the Yahboom docs
- Mapping (SLAM) and Nav2 navigation
- LiDAR follow and patrol, line following, color tracking
- AI functions: object recognition and voice or text commands



# 01

# What is ROS 2?

The ideas behind every ROS robot, including ours.

# Every robot does three things



## Sense

LiDAR, camera, IMU



## Think

Raspberry Pi or Jetson



## Act

Motors and steering servo

*The hard part is not each piece. It is getting all the pieces to talk to each other.*

# In ROS 2, each part is a program called a node

**ROS 2 is not an operating system.** It is a set of tools and libraries that runs on Linux and lets small programs on a robot send data to each other.




**Why this matters:** you can swap one node without touching the others. New LiDAR? Change the sense node. Better algorithm? Change the think node. The motor node does not care.

# Nodes talk through topics

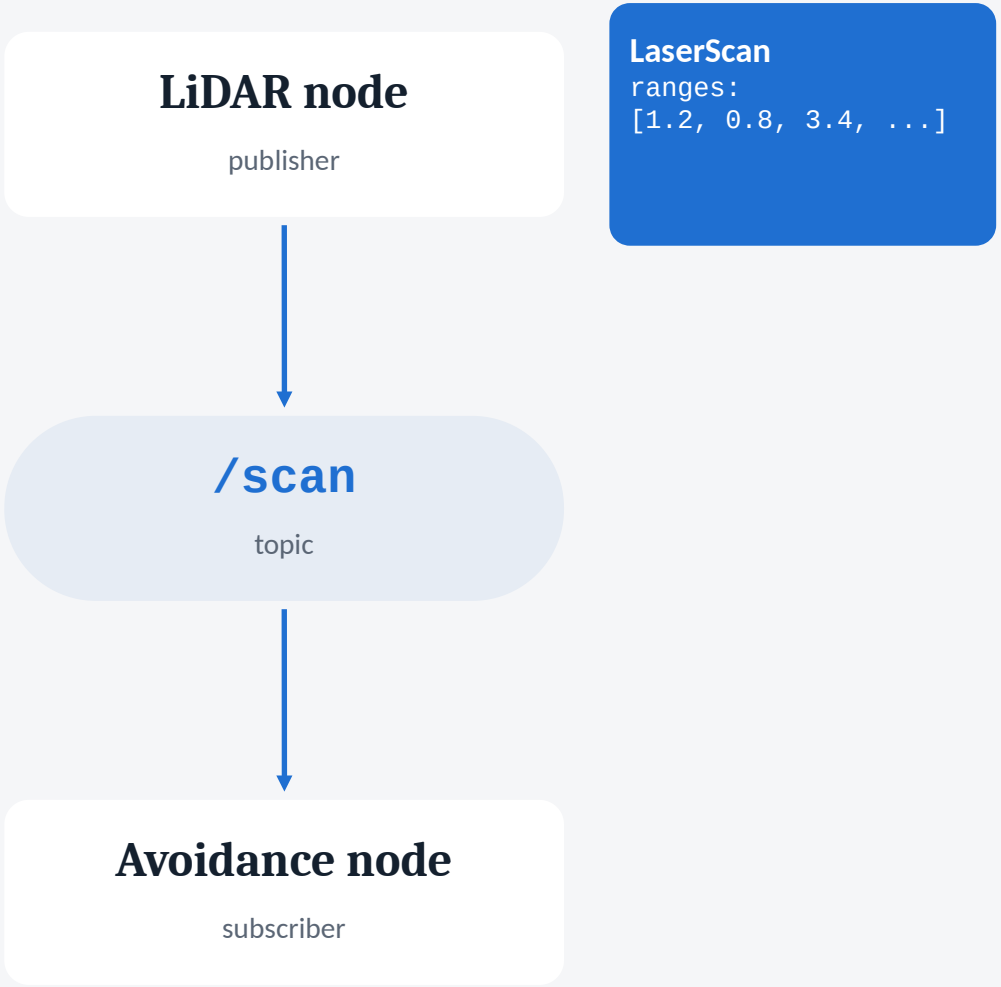
**Node** — A program that does one job.

**Topic** — A named channel, like /scan. Anyone can listen.

**Message** — The data sent on a topic, with a fixed type.

**Publisher** — A node that sends messages to a topic.

**Subscriber** — A node that receives messages from a topic.



# Nodes talk through topics

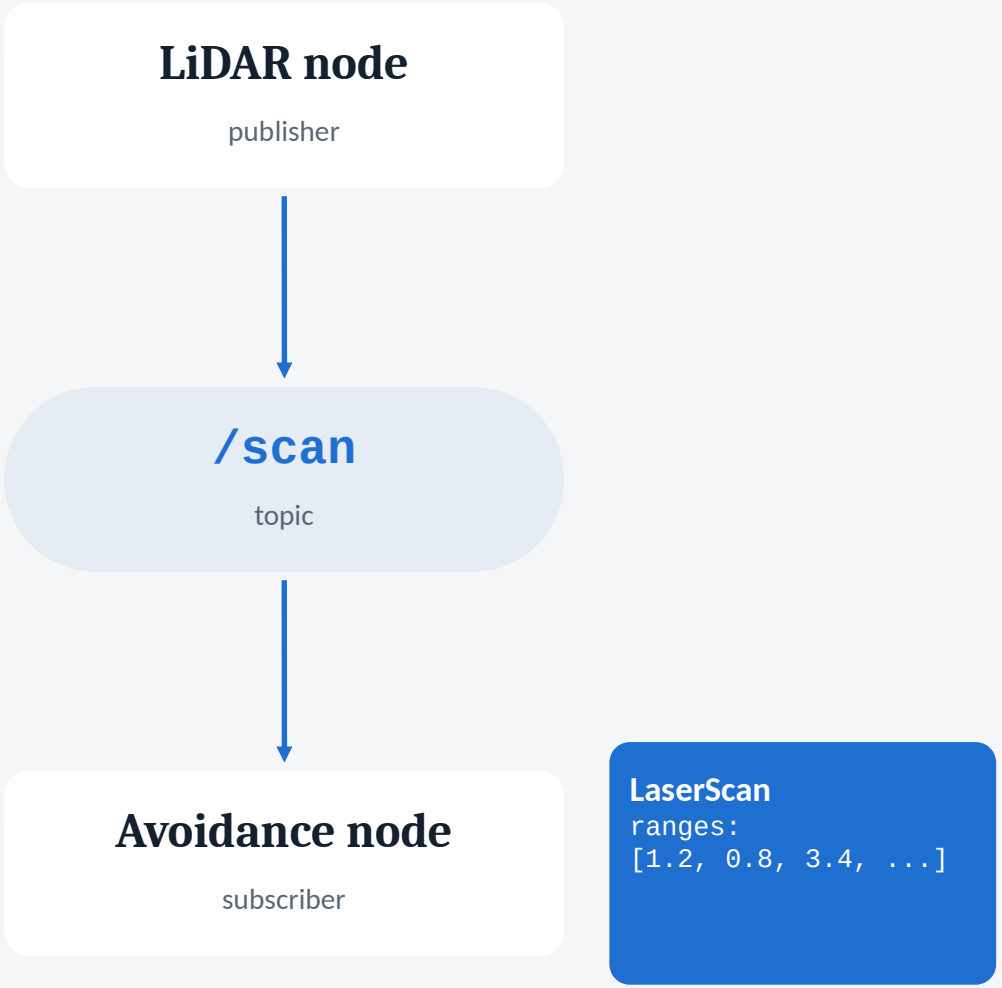
**Node** — A program that does one job.

**Topic** — A named channel, like /scan. Anyone can listen.

**Message** — The data sent on a topic, with a fixed type.

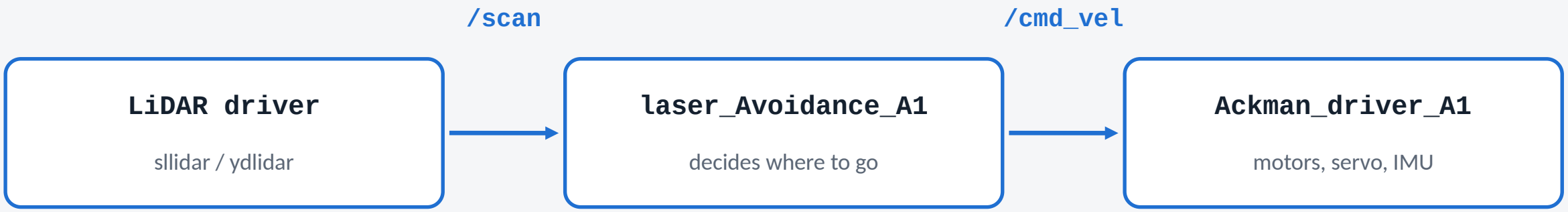
**Publisher** — A node that sends messages to a topic.

**Subscriber** — A node that receives messages from a topic.



OUR ROBOT

# The ROSMASTER A1 is a ROS 2 graph






These are the real node names on the A1. In Week 3 you start them yourself:

```
# start the chassis (motors + servo)
ros2 run yahboomcar_bringup Ackman_driver_A1
# start the LiDAR (C1 model)
ros2 launch sllidar_ros2 sllidar_c1_launch.py
# start obstacle avoidance
ros2 run yahboomcar_laser laser_Avoidance_A1
```

OUR ROBOT

# Topics you will see on the A1

| Topic | Message type | What it carries |
|---|---|---|
| `/scan` | `sensor_msgs/LaserScan` | LiDAR distances, 360 degrees |
| `/cmd_vel` | `geometry_msgs/Twist` | Speed and turning command to the robot |
| `/odom` | `nav_msgs/Odometry` | Where the robot thinks it is |
| `/imu` | `sensor_msgs/Imu` | Rotation and acceleration |
| `/map` | `nav_msgs/OccupancyGrid` | 2D map: free, wall, or unknown |
| `/camera/image_raw` | `sensor_msgs/Image` | Camera picture |

**Today we only need the first two: read /scan, write /cmd_vel.**

VERSIONS

# ROS and ROS 2 are different systems

## ROS 1

- Started in 2007
- Needs one central master (roscore). If it dies, everything stops.
- Built for one robot on a good network
- Last version Noetic. End of life May 31, 2025.

## ROS 2

- First release December 2017
- No master. Nodes find each other on the network by themselves (DDS).
- Built for many robots, Wi-Fi, and real products
- Actively developed. The A1 runs ROS 2.

VERSIONS

# ROS 2 has a new version every year




## Newest: Lyrical Luth

**Released May 22, 2026. Supported until May 2031.**

Each year a new version comes out on May 23, World Turtle Day. Even years are LTS (long-term support, 5 years).

VERSIONS

# ROS 2 has a new version every year




## We use Humble Hawksbill

**Ubuntu 22.04 · supported until May 2027**

The ROSMASTER A1 ships with Humble, and our lab containers use it too. Each version is tied to one Ubuntu version, so match them when you install.

HOW CODE IS ORGANIZED

# A workspace is just a folder



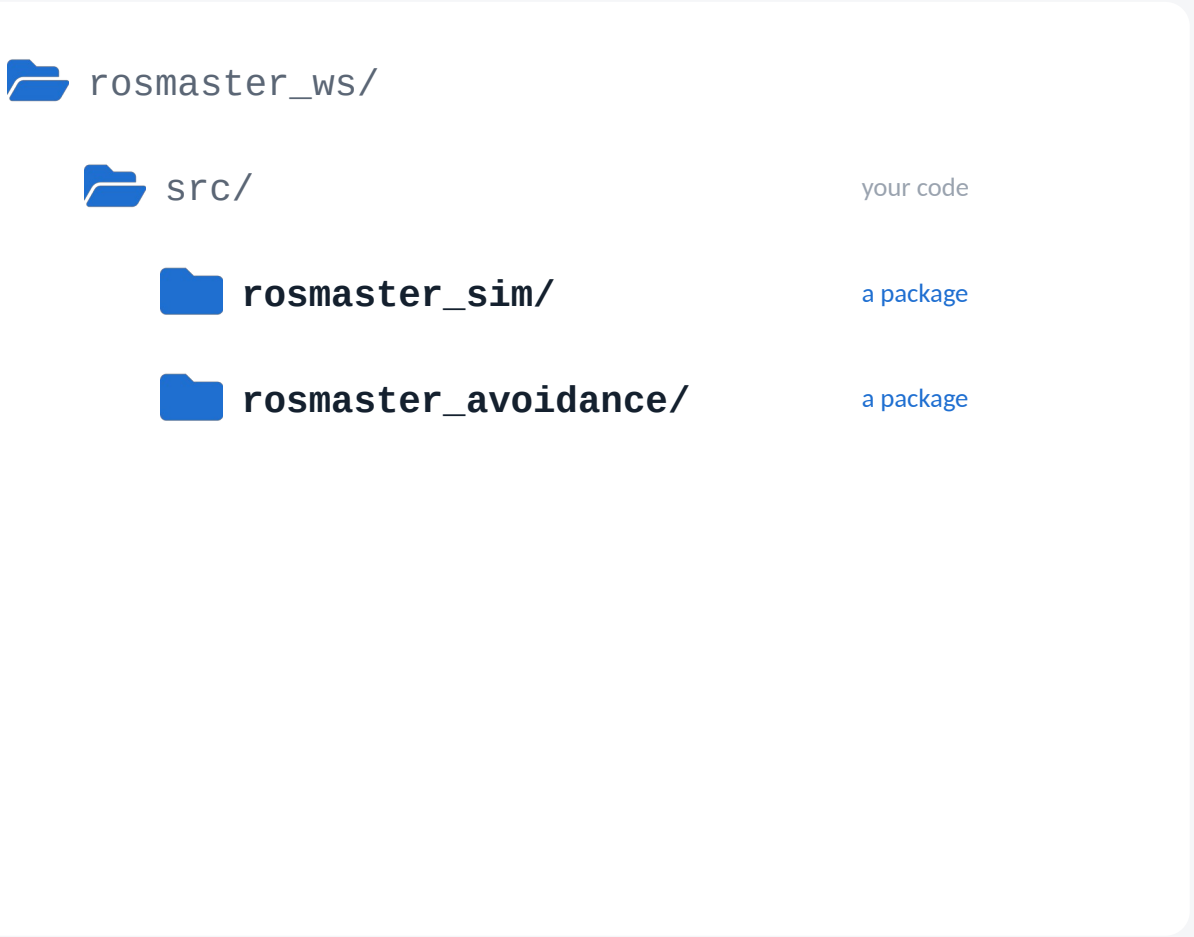

Everything for one robot project lives in one folder: the workspace.

Inside it, src/ (source) is where all your code goes. You can name the workspace anything.

```
# make both folders at once
mkdir -p ~/rosmaster_ws/src
```

HOW CODE IS ORGANIZED

# Inside src/ live packages




**A package is a folder that does one job.** ROS 2 builds, installs and runs your code one package at a time.

*Think of the workspace as your phone, and packages as the apps on it.*

### `rosmaster_sim`

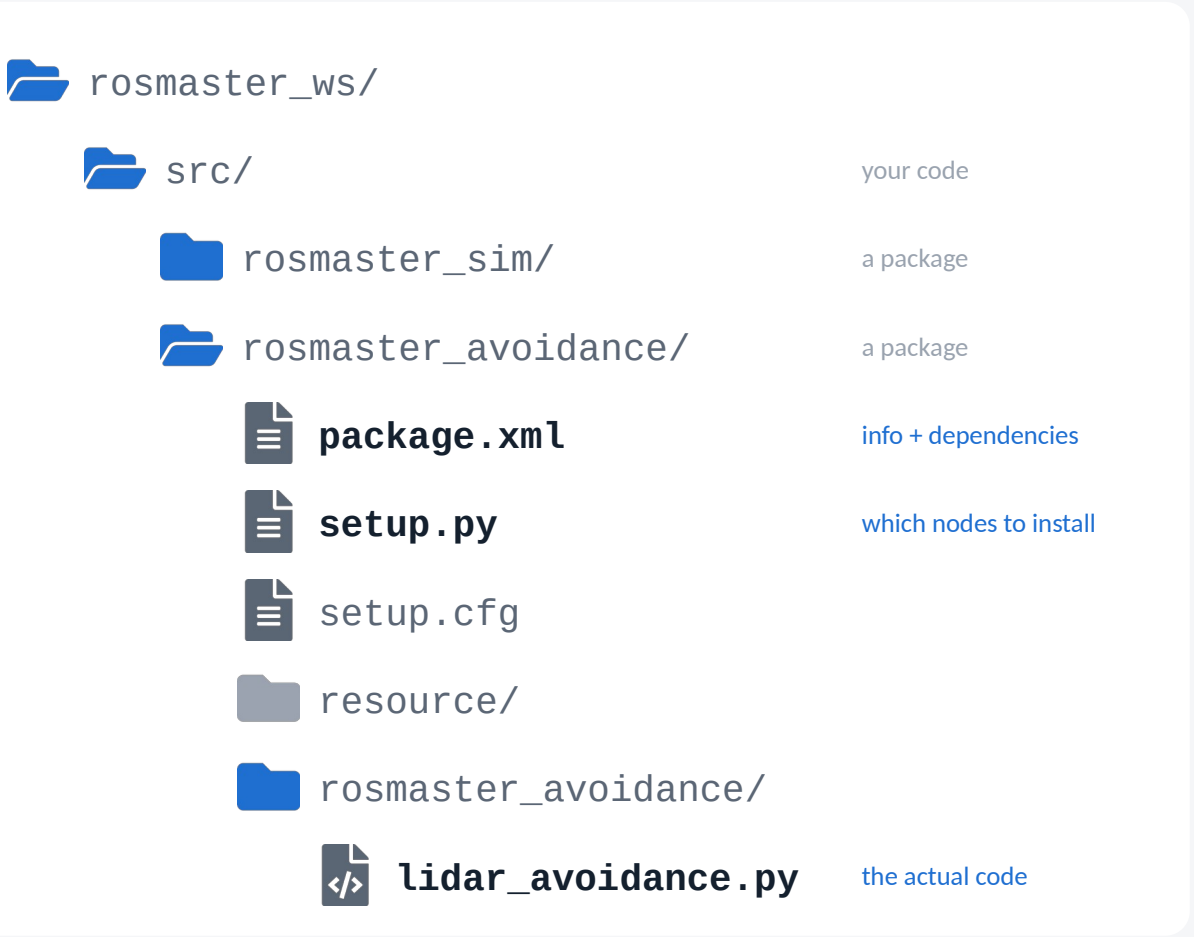
The A1 robot model, the Gazebo world, and the launch file that starts them.

### `rosmaster_avoidance`

Our Python node: reads /scan, publishes /cmd_vel.

HOW CODE IS ORGANIZED

# What is inside a package




## `package.xml`

The ID card: name, version, and which other packages it needs (rclpy, sensor_msgs…).

## `setup.py`

Tells ROS which nodes this package has, so ros2 run can find them.

## `lidar_avoidance.py`

The real code. This is the file we read in Week 2.

You do not write these by hand. One command makes the whole package:

```
ros2 pkg create --build-type ament_python \
  rosmaster_avoidance
```

HOW CODE IS ORGANIZED

# Build, source, run




colcon reads every package in src/ and makes three new folders. You never edit them.

**1 Build**

`colcon build`

**2 Source**

`source install/setup.bash`

**3 Run**

`ros2 run / ros2 launch`

**Forgot to source? ROS says "package not found". Every new terminal needs it.**

# Commands you will use today

```
ros2 run <package> <node>          # start one node
ros2 launch <package> <file>       # start many nodes at once

ros2 node list                     # which nodes are running?
ros2 topic list                    # which topics exist?
ros2 topic echo /scan              # print messages live
ros2 topic hz /scan                # how fast are they coming?

rqt_graph                          # draw the node graph
```

**Stuck? Press Ctrl+C to stop a running node.**

# 02

# Gazebo

Test the robot in a virtual world before touching the real one.

GAZEBO

# A virtual world with real physics

## Physics

Gravity, friction and collisions behave like the real world.

## Virtual sensors

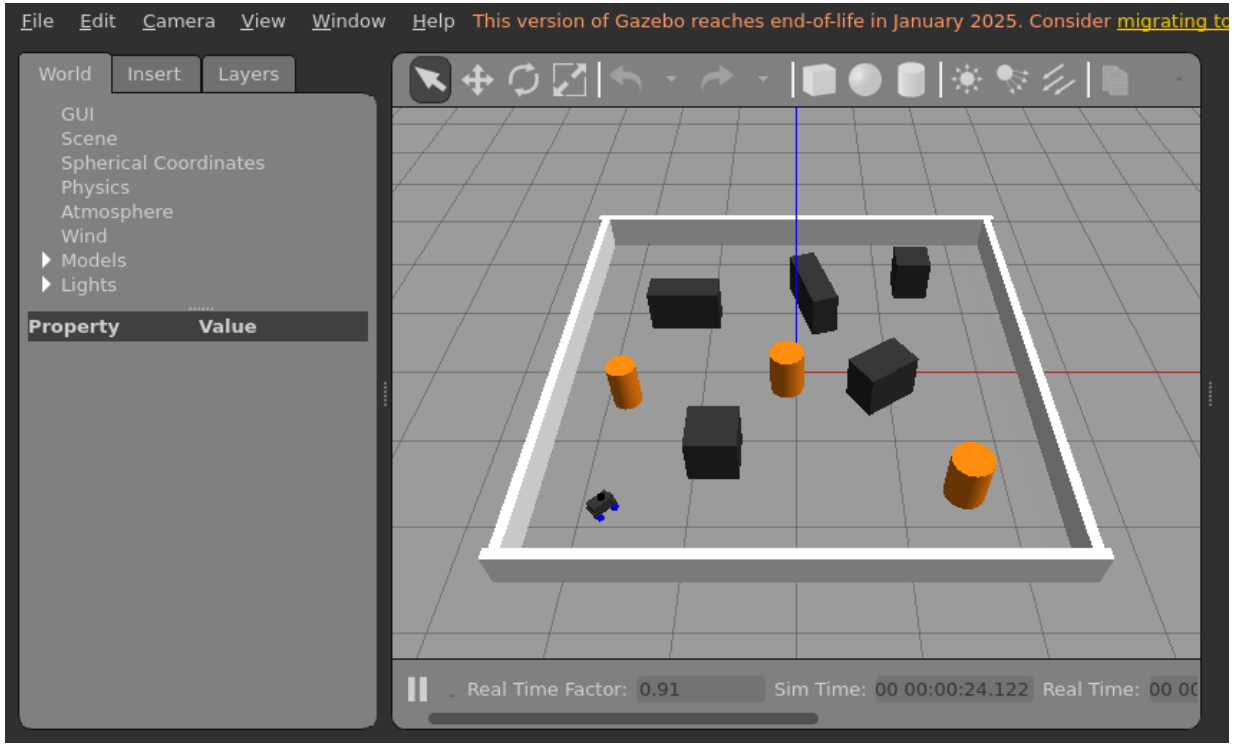
A simulated LiDAR, camera or IMU produces realistic data.

## Same topics

Gazebo plugins publish /scan and listen to /cmd_vel, just like the real robot.

## Free crashes

Hit a wall a hundred times. Nothing breaks, no batteries die.



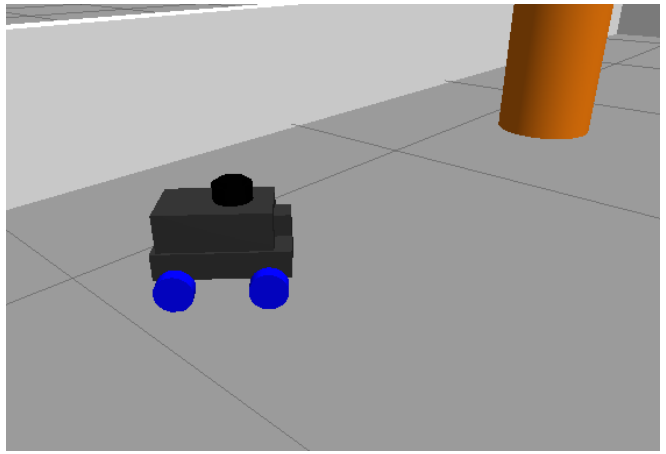

Our room and our A1 model in Gazebo Classic 11, which comes with ROS 2 Humble. Classic reached end of life in January 2025; the newer Gazebo (Harmonic and later) is the next step after this course.

GAZEBO

# Same node. Simulation or real robot.

## Gazebo

Week 1 and 2



**`lidar_avoidance`**

your node, unchanged

`/scan in`
`/cmd_vel out`

## Real ROSMASTER A1

Week 3 and 4

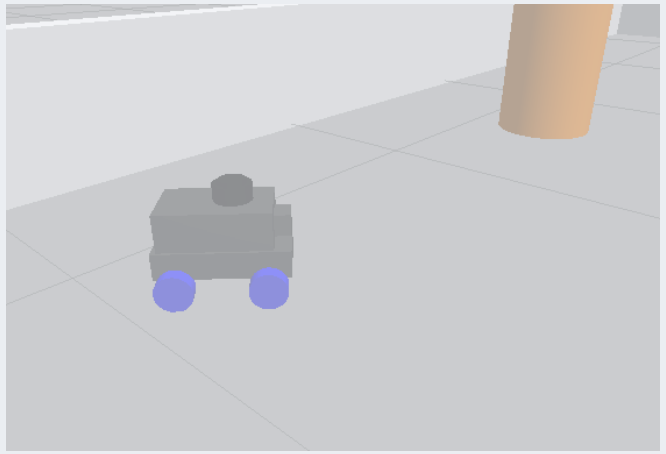


GAZEBO

# Same node. Simulation or real robot.

## Gazebo

Week 1 and 2



**`lidar_avoidance`**

your node, unchanged

`/scan in`
`/cmd_vel out`

## Real ROSMASTER A1

Week 3 and 4



# 03

# Hands-on

Make the simulated A1 drive around without hitting anything.

SET UP YOUR PC

# Install on Windows, step by step

1. Install Ubuntu (WSL2) — 2. Prepare Ubuntu — 3. Add the ROS 2 source — 4. Install ROS 2 + Gazebo — 5. Load ROS 2 + test — 6. Test Gazebo

**Windows PowerShell (Run as administrator)**

```
# Right click PowerShell > Run as administrator
wsl --install -d Ubuntu-22.04

# Restart the PC. Open "Ubuntu 22.04" from the Start menu.
# Make a username and password. The password is invisible while you type.

# Check, inside Ubuntu:
lsb_release -a          # should say Ubuntu 22.04
```

## What this does

WSL2 runs a real Ubuntu 22.04 inside Windows. ROS 2 Humble needs Ubuntu 22.04.

**Watch out**

Needs Windows 11, or Windows 10 build 19044+ (press Win+R, type winver). Already have WSL? Run wsl --update first.

SET UP YOUR PC

# Install on Windows, step by step

1. Install Ubuntu (WSL2) — **2. Prepare Ubuntu** — 3. Add the ROS 2 source — 4. Install ROS 2 + Gazebo — 5. Load ROS 2 + test — 6. Test Gazebo

**Ubuntu terminal**

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y locales software-properties-common curl git

# ROS 2 needs a UTF-8 language setting
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

sudo add-apt-repository -y universe
```

### What this does

Updates Ubuntu, installs a few tools we need (curl, git), sets the language to UTF-8, and turns on Ubuntu's universe packages.

### Watch out

sudo asks for your Ubuntu password. Type it and press Enter, even though nothing shows.

SET UP YOUR PC

# Install on Windows, step by step

1. Install Ubuntu (WSL2) — 2. Prepare Ubuntu — **3. Add the ROS 2 source** — 4. Install ROS 2 + Gazebo — 5. Load ROS 2 + test — 6. Test Gazebo

**Ubuntu terminal**

```
export ROS_APT_SOURCE_VERSION=$(curl -s \
  https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest \
  | grep -F "tag_name" | awk -F'"' '{print $4}')
V=$ROS_APT_SOURCE_VERSION

curl -L -o /tmp/ros2-apt-source.deb \
  https://github.com/ros-infrastructure/ros-apt-source/releases/download/$V/ros2-apt-source_$V.jammy_all.deb

sudo dpkg -i /tmp/ros2-apt-source.deb
```

**What this does**

apt does not know where ROS 2 is yet. This downloads a small package that adds the official ROS 2 download source.

**Watch out**

Copy it exactly, or copy from the README. This is the method in the official ROS 2 Humble install guide.

SET UP YOUR PC

# Install on Windows, step by step

1. Install Ubuntu (WSL2) — 2. Prepare Ubuntu — 3. Add the ROS 2 source — **4. Install ROS 2 + Gazebo** — 5. Load ROS 2 + test — 6. Test Gazebo

**Ubuntu terminal**

```
sudo apt update && sudo apt upgrade -y

# ROS 2 Humble with RViz and demos, plus build tools (colcon)
sudo apt install -y ros-humble-desktop ros-dev-tools

# Gazebo + its ROS plugins, robot model tools, keyboard driving
sudo apt install -y ros-humble-gazebo-ros-pkgs ros-humble-xacro \
  ros-humble-teleop-twist-keyboard
```

**What this does**

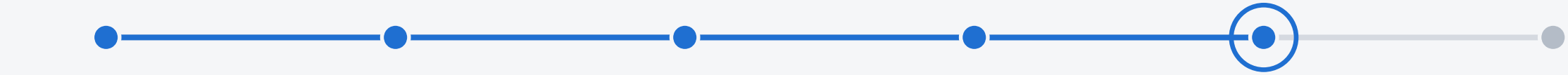
Installs everything the course uses: ROS 2 Humble, colcon, Gazebo 11 and its ROS plugins, xacro, and keyboard teleop.

**Watch out**

A few GB. It can take 20 to 40 minutes. Do it at home on fast Wi-Fi, before class.

SET UP YOUR PC

# Install on Windows, step by step




**Ubuntu terminal**

```
# Load ROS 2 in every new terminal automatically
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc

# Test. Terminal 1:
ros2 run demo_nodes_cpp talker

# Terminal 2 (open a second Ubuntu window):
ros2 run demo_nodes_py listener
```

**What this does**

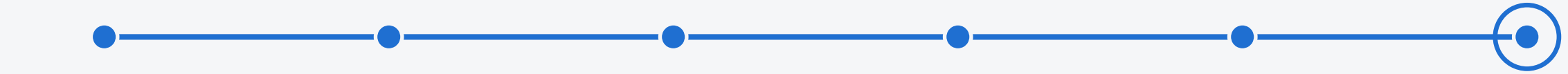
~/.bashrc runs every time a terminal opens, so ROS 2 is always ready. The talker and listener are two nodes talking on a topic.

**Watch out**

Listener prints "I heard: [Hello World: 1]"? ROS 2 works. Stop both with Ctrl+C.

SET UP YOUR PC

# Install on Windows, step by step




**Ubuntu terminal**

```
gazebo
# An empty Gazebo world opens on your Windows desktop. Close it.

# Black window or crash? Use software graphics:
echo "export LIBGL_ALWAYS_SOFTWARE=1" >> ~/.bashrc
source ~/.bashrc
```

### What this does

WSL2 shows Linux windows (WSLg) right on your Windows desktop, so Gazebo opens like a normal app.

**Watch out**

First start is slow. Update your Windows graphics driver (Intel, AMD or NVIDIA) for smoother Gazebo.

HANDS-ON

# Run it yourself, step by step

```
# 1. Get the code
mkdir -p ~/rosmaster_ws/src && cd ~/rosmaster_ws/src
git clone https://github.com/Phearamony/ROSMASTER_crash_course.git

# 2. Build it
cd ~/rosmaster_ws && colcon build --symlink-install
source install/setup.bash

# 3. Terminal 1: open Gazebo with the A1
ros2 launch rosmaster_sim sim.launch.py

# 4. Terminal 2 (source first!): start the brain
ros2 run rosmaster_avoidance lidar_avoidance
```

Step 1

## Download the course

mkdir makes the workspace and its src/ folder. git clone copies both course packages from GitHub into src/.

After this, src/ holds rosmaster_sim and rosmaster_avoidance.

HANDS-ON

# Run it yourself, step by step

```
# 1. Get the code
mkdir -p ~/rosmaster_ws/src && cd ~/rosmaster_ws/src
git clone https://github.com/Phearamony/ROSMASTER_crash_course.git

# 2. Build it
cd ~/rosmaster_ws && colcon build --symlink-install
source install/setup.bash

# 3. Terminal 1: open Gazebo with the A1
ros2 launch rosmaster_sim sim.launch.py

# 4. Terminal 2 (source first!): start the brain
ros2 run rosmaster_avoidance lidar_avoidance
```

Step 2

## Build and source

colcon build turns the packages into programs inside install/. --symlink-install links your .py files, so a code change works without rebuilding.

source install/setup.bash tells this terminal where those programs are.

HANDS-ON

# Run it yourself, step by step

```
# 1. Get the code
mkdir -p ~/rosmaster_ws/src && cd ~/rosmaster_ws/src
git clone https://github.com/Phearamony/ROSMASTER_crash_course.git

# 2. Build it
cd ~/rosmaster_ws && colcon build --symlink-install
source install/setup.bash

# 3. Terminal 1: open Gazebo with the A1
ros2 launch rosmaster_sim sim.launch.py

# 4. Terminal 2 (source first!): start the brain
ros2 run rosmaster_avoidance lidar_avoidance
```

Step 3

## Open the simulation

One launch file starts three things: Gazebo with the obstacle room, the A1 model with its LiDAR, and a node that publishes the robot's shape.

The robot sits still. It is publishing /scan, and waiting for someone to send /cmd_vel.

HANDS-ON

# Run it yourself, step by step

```
# 1. Get the code
mkdir -p ~/rosmaster_ws/src && cd ~/rosmaster_ws/src
git clone https://github.com/Phearamony/ROSMASTER_crash_course.git

# 2. Build it
cd ~/rosmaster_ws && colcon build --symlink-install
source install/setup.bash

# 3. Terminal 1: open Gazebo with the A1
ros2 launch rosmaster_sim sim.launch.py

# 4. Terminal 2 (source first!): start the brain
ros2 run rosmaster_avoidance lidar_avoidance
```

Step 4

## Start the brain

Our node subscribes to /scan, decides 10 times per second, and publishes /cmd_vel. The robot starts driving.

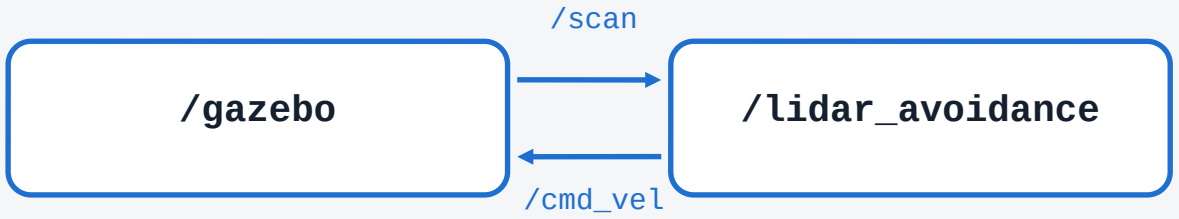
Terminal shows what it decides: FORWARD front=2.31 left=1.05 right=0.88

HANDS-ON

# Is it working? Check with ROS 2 tools

```
# Terminal 3 (source first!)
ros2 node list             # see lidar_avoidance?
ros2 topic list            # see /scan and /cmd_vel?
ros2 topic echo /cmd_vel   # the speeds our node sends
rqt_graph                  # draw the graph
```

rqt_graph should show the same picture as earlier in this deck:



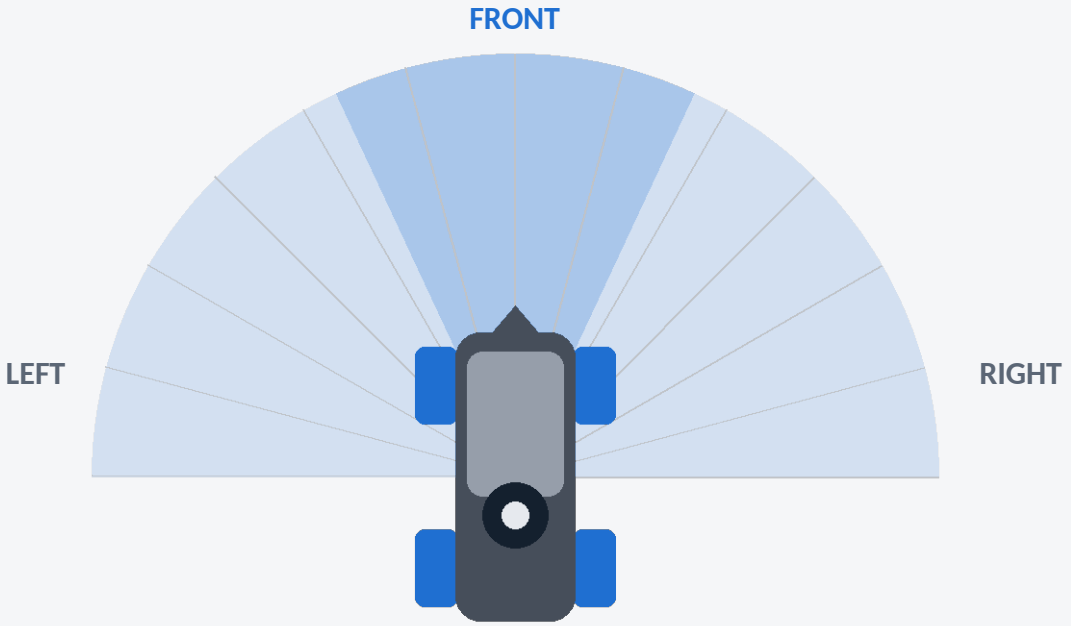

**Course repo**

github.com/Phearamony/ROSMASTER_crash_course

**Full guide + fixes in the README**

HANDS-ON

# How the robot sees: 360 LiDAR distances




The LiDAR spins and measures the distance to the nearest thing in every direction, about 10 times per second. That list of numbers is the /scan message.

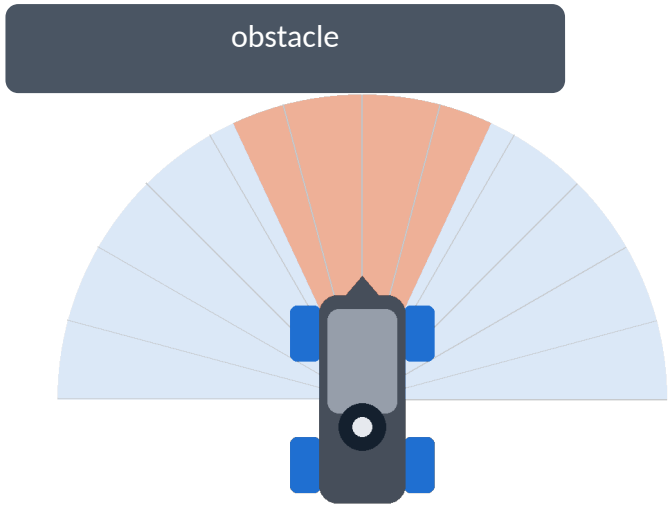
Our node splits it into three zones and keeps only the closest distance in each:

| | |
|---|---|
| `front` | -25° to 25° |
| `left` | 25° to 90° |
| `right` | -90° to -25° |

*0° is straight ahead. Positive angles are to the left.*

HANDS-ON

# Three rules, checked 10 times per second




1 **Front is clear**

Drive forward.

2 **Front is blocked**

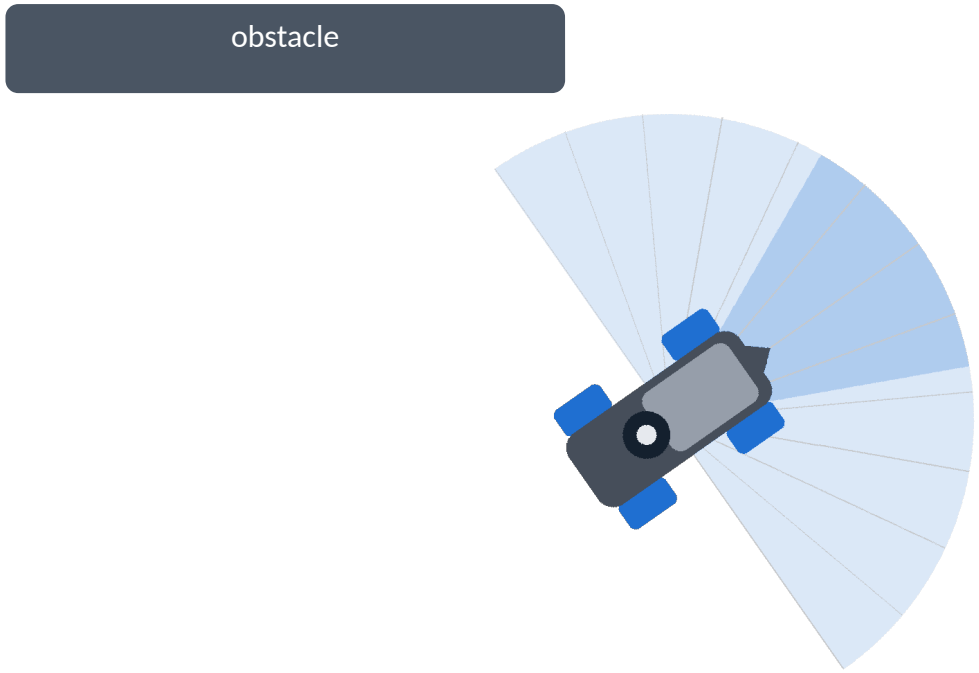
Slow down and steer toward the side with more space.

3 **Too close**

Back up for a moment, then try again.

HANDS-ON

# Three rules, checked 10 times per second




**1** **Front is clear**

Drive forward.

**2** **Front is blocked**

Slow down and steer toward the side with more space.

**3** **Too close**

Back up for a moment, then try again.

HANDS-ON

# Inside lidar_avoidance.py

```
class LidarAvoidance(Node):
    def __init__(self):
        super().__init__('lidar_avoidance')
        self.declare_parameter('forward_speed', 0.25)
        self.declare_parameter('safe_distance', 0.7)
        ...
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cmd_pub = self.create_publisher(
            Twist, '/cmd_vel', 10)

    def scan_callback(self, scan):
        front = self.sector_min(scan, -25, 25)
        left = self.sector_min(scan, 25, 90)
        right = self.sector_min(scan, -90, -25)

        cmd = Twist()
        if front < stop:          # rule 3: back up
            cmd.linear.x = -0.15
        elif front < safe:        # rule 2: turn
            cmd.linear.x = forward_speed * 0.6
            cmd.angular.z = turn_speed if left > right else -turn_speed
        else:                     # rule 1: go
            cmd.linear.x = forward_speed
        self.cmd_pub.publish(cmd)
```

*Simplified. The full file is in the repo.*

Part 1 of 4

## Listen and talk

The node subscribes to /scan and creates a publisher for /cmd_vel. Every time a scan arrives, ROS calls scan_callback for us.

We never write a loop that waits for data. ROS 2 calls our function.

HANDS-ON

# Inside lidar_avoidance.py

```
class LidarAvoidance(Node):
    def __init__(self):
        super().__init__('lidar_avoidance')
        self.declare_parameter('forward_speed', 0.25)
        self.declare_parameter('safe_distance', 0.7)
        ...
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cmd_pub = self.create_publisher(
            Twist, '/cmd_vel', 10)

    def scan_callback(self, scan):
        front = self.sector_min(scan, -25, 25)
        left = self.sector_min(scan, 25, 90)
        right = self.sector_min(scan, -90, -25)

        cmd = Twist()
        if front < stop:          # rule 3: back up
            cmd.linear.x = -0.15
        elif front < safe:        # rule 2: turn
            cmd.linear.x = forward_speed * 0.6
            cmd.angular.z = turn_speed if left > right else -turn_speed
        else:                     # rule 1: go
            cmd.linear.x = forward_speed
        self.cmd_pub.publish(cmd)
```

Part 2 of 4

## Knobs you can turn

Parameters are settings with a default value. You change them from the terminal without editing code.

```
--ros-args -p
safe_distance:=1.0
```

HANDS-ON

# Inside lidar_avoidance.py

```
class LidarAvoidance(Node):
    def __init__(self):
        super().__init__('lidar_avoidance')
        self.declare_parameter('forward_speed', 0.25)
        self.declare_parameter('safe_distance', 0.7)
        ...
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cmd_pub = self.create_publisher(
            Twist, '/cmd_vel', 10)

    def scan_callback(self, scan):
        front = self.sector_min(scan, -25, 25)
        left = self.sector_min(scan, 25, 90)
        right = self.sector_min(scan, -90, -25)

        cmd = Twist()
        if front < stop:          # rule 3: back up
            cmd.linear.x = -0.15
        elif front < safe:        # rule 2: turn
            cmd.linear.x = forward_speed * 0.6
            cmd.angular.z = turn_speed if left > right else -turn_speed
        else:                     # rule 1: go
            cmd.linear.x = forward_speed
        self.cmd_pub.publish(cmd)
```

Part 3 of 4

## Look

sector_min goes through all 360 distances and returns the closest one inside each zone.

```
front=0.62  left=1.80
right=0.45
```

HANDS-ON

# Inside lidar_avoidance.py

```
class LidarAvoidance(Node):
    def __init__(self):
        super().__init__('lidar_avoidance')
        self.declare_parameter('forward_speed', 0.25)
        self.declare_parameter('safe_distance', 0.7)
        ...
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cmd_pub = self.create_publisher(
            Twist, '/cmd_vel', 10)

    def scan_callback(self, scan):
        front = self.sector_min(scan, -25, 25)
        left = self.sector_min(scan, 25, 90)
        right = self.sector_min(scan, -90, -25)

        cmd = Twist()
        if front < stop:          # rule 3: back up
            cmd.linear.x = -0.15
        elif front < safe:        # rule 2: turn
            cmd.linear.x = forward_speed * 0.6
            cmd.angular.z = turn_speed if left > right else -turn_speed
        else:                     # rule 1: go
            cmd.linear.x = forward_speed
        self.cmd_pub.publish(cmd)
```

Part 4 of 4

## Decide and act

A Twist message holds the command. linear.x is forward speed (m/s). angular.z is turning (rad/s, positive = left). Then we publish it.

The A1 steers like a car, so angular.z becomes a steering angle.

HANDS-ON

# Your turn: four small experiments

## 1 Drive it yourself

Stop the avoidance node, then:

```
ros2 run teleop_twist_keyboard
teleop_twist_keyboard
```

## 2 Look at the data

Move the robot near a box and watch the numbers:

```
ros2 topic echo /scan --once
```

## 3 Change its behavior

Make it more careful:

```
ros2 run rosmaster_avoidance lidar_avoidance \
    --ros-args -p safe_distance:=1.0
```

## 4 Make it harder

In Gazebo, use the Insert tab or the box tool to add obstacles. Does the robot still escape?

NEXT WEEK

# Week 2: read every line of the code you just ran

```
def scan_callback(self, scan):
    front = self.sector_min(scan, -25, 25)
    left = self.sector_min(scan, 25, 90)
    right = self.sector_min(scan, -90, -25)

    cmd = Twist()
    if front < safe:
        cmd.linear.x = forward_speed * 0.6
        cmd.angular.z = turn_speed if left > right else -turn_speed
```

## Python

variables, if, functions, classes

## Bash

cd, ls, nano, source, sudo apt

## Linux

files, folders, permissions, the terminal

**Homework: run the four experiments. Bring one question.**